BCSE301P

Software Engineering Lab

Assignment 14: Test Report

Project: MomentsHub (v1.0)

Name: G.Sasmith

Reg. No: 23BPS1074

Date: November 7, 2025

Faculty: Dr. Softya Sebastian

Slot: L5+L6

1. Introduction

This document presents the results of functional and non-functional testing performed on the MomentsHub web application (v1.0), as required by Assignment 14. The primary objective is to verify that the application's core features align with the Software Requirements Specification (SRS) and to audit the application's real-world performance, accessibility, and best practices.

2. Testing Methodology & Tools

2.1 Functional Testing Tool: Cypress

To verify core functionality, the Cypress E2E (End-to-End) testing framework was used. Cypress was chosen because it is a modern, all-in-one testing tool designed specifically for JavaScript applications like React. It runs tests in a real browser to simulate user stories such as logging in, uploading photos, and commenting.

2.2 Non-Functional Testing Tool: Google Lighthouse

To audit non-functional requirements, Google Lighthouse was used. Lighthouse is an open-source tool built into Chrome that provides a comprehensive analysis of an application's **Performance**, **Accessibility**, **Best Practices**, and **SEO**.

3. Functional Testing Results

All functional tests were executed using Cypress. The application was run locally (http://localhost:5173) and tests were performed in a Chrome browser controlled by Cypress.

3.1 Test Cases & Results

Test ID	Requirement	Test Case	Expected Result	Actual Result	Status
TC-01	FR1.2	**User Login:** A user with valid credentials attempts to log in.	The user is redirected to the dashboard and sees the "Today's Moments" title.	User was logged in successfully and redirected to the dashboard. The title was visible.	**PASS**
TC-02	FR2.1 FR3.5	**Photo Upload with Tag:** A logged-in user uploads a photo, adds a caption, and adds a tag.	The photo modal closes. The new photo appears on the dashboard with the correct caption and	The photo appeared instantly on the dashboard with the correct caption and the "#test-	**PASS**

			tag (e.g., "#test-tag").	tag" was visible.	
TC-03	FR2.4	**Add Comment:** A user opens a photo's detail page and posts a comment.	The new comment appears in the comment list immediately without a page reload.	The comment was successfully posted and appeared in the list.	**PASS**

3.2 Functional Test Evidence:

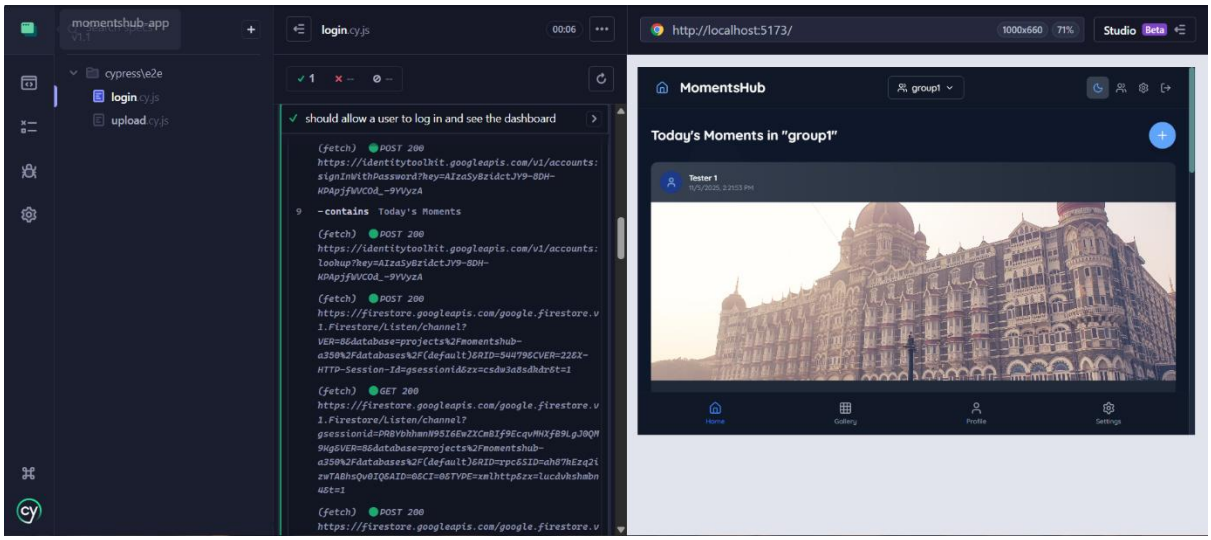


Figure 1: TC-01 (Login) Test Case Pass

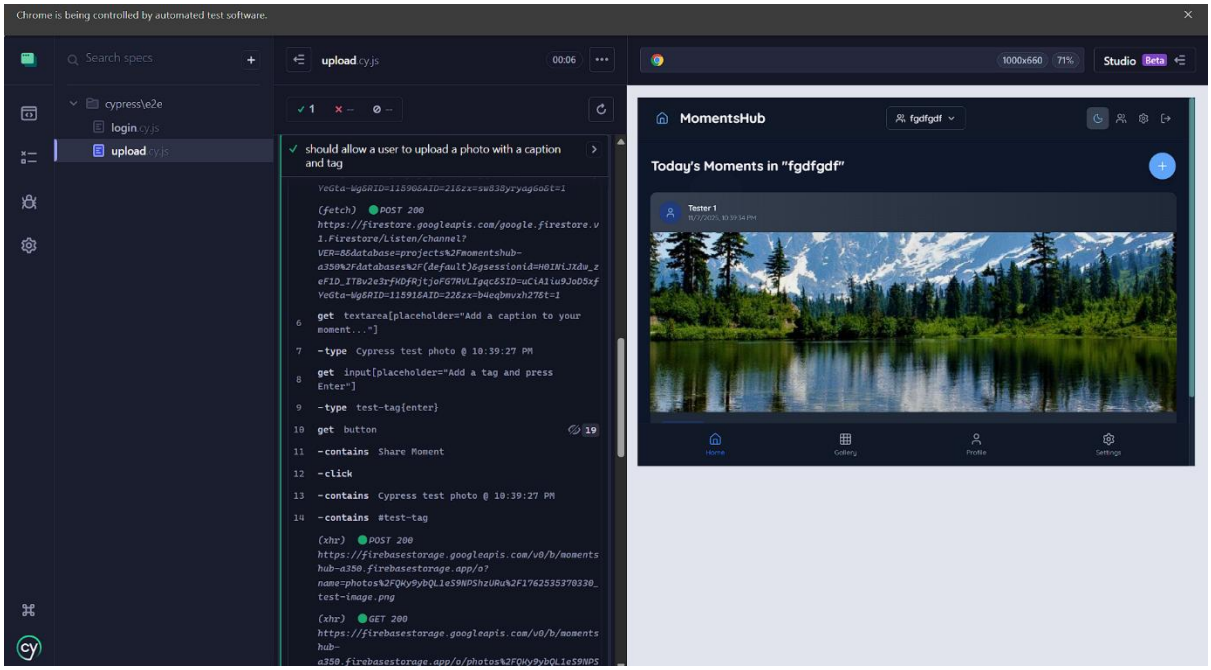


Figure 2: TC-02 (Upload w/ Tag) Test Case Pass

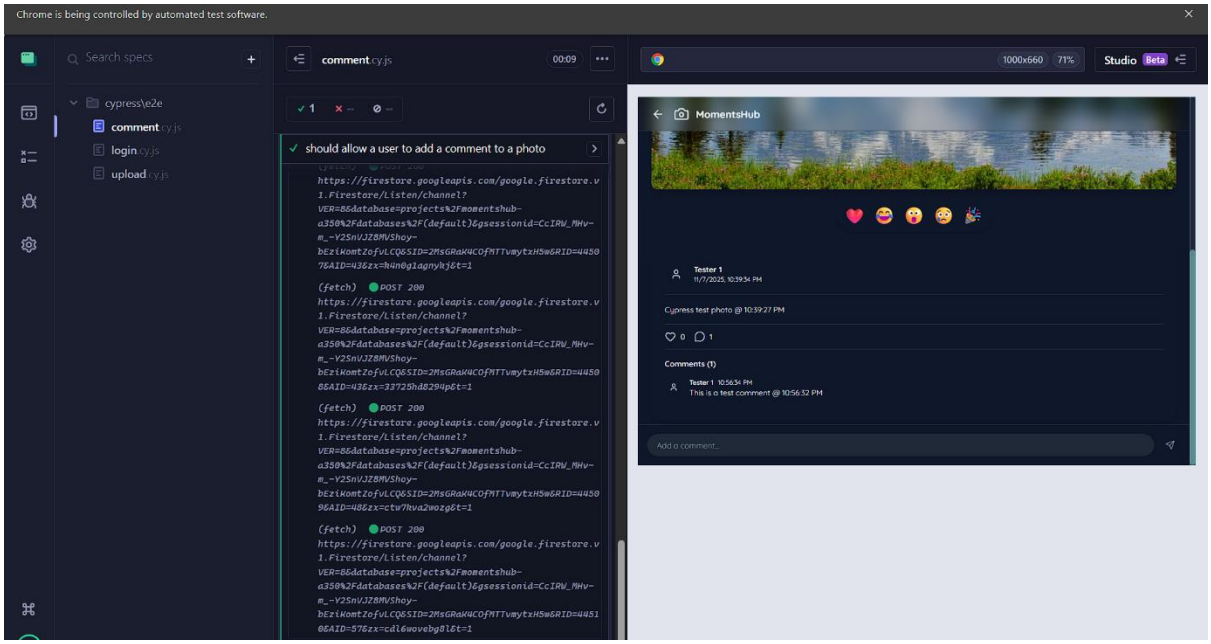
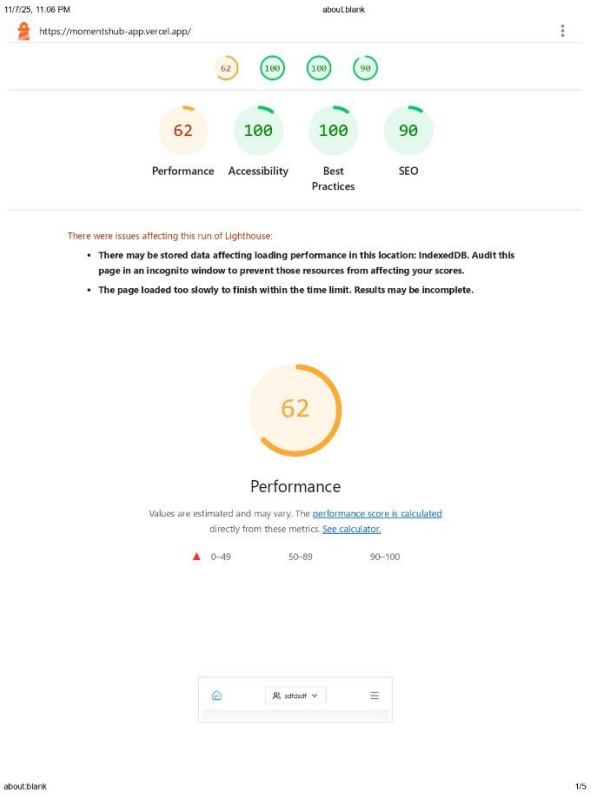


Figure 3: TC-03 (Add Comment) Test Case Pass

4. Non-Functional Testing Results

The non-functional audit was performed on the deployed Vercel application URL (https://momentshub-app.vercel.app/). The test was run simulating a **Desktop** device to establish a baseline for performance and quality.

4.1 Lighthouse Score Summary



METRICS Expand view

▲ First Contentful Paint
4.7 s

▲ Largest Contentful Paint
5.6 s

Total Blocking Time
0 ms

Cumulative Layout Shift
0

▲ Speed Index
9.0 s

Later this year, insights will replace performance audits. Learn more and provide feedback here.

Go back to audits

Show audits relevant to: All FCP LCP TBT

INSIGHTS

- ▲ Use efficient cache lifetimes — Est savings of 78 KiB
- ▲ Render blocking requests — Est savings of 150 ms
- ▲ Network dependency tree
- ○ LCP breakdown
- ○ 3rd parties

These insights are also available in the Chrome DevTools Performance Panel - record a trace to view more detailed information.

DIAGNOSTICS

- ▲ Minify JavaScript — Est savings of 41 KiB
- ▲ Reduce unused JavaScript — Est savings of 88 KiB

- Avoid long main-thread tasks — 5 long tasks found

More information about the performance of your application. These numbers don't directly affect the Performance score.

PASSED AUDITS (25) Show

100

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

ADDITIONAL ITEMS TO MANUALLY CHECK (10) Show

These items address areas which an automated testing tool cannot cover. Learn more in our guide on conducting an accessibility review.

PASSED AUDITS (15) Show

NOT APPLICABLE (42) Show

100

Best Practices

TRUST AND SAFETY

- Ensure CSP is effective against XSS attacks
- Ensure proper origin isolation with COOP
- Mitigate clickjacking with XFO or CSP

Mitigate DOM-based XSS with Trusted Types

GENERAL

Missing source maps for large first-party JavaScript

PASSED AUDITS (14) Show

NOT APPLICABLE (3) Show

90

SEO

These checks ensure that your page is following basic search engine optimization advice. There are many additional factors Lighthouse does not score here that may affect your search ranking, including performance on Core Web Vitals. Learn more about Google Search Essentials.

CONTENT BEST PRACTICES

Document does not have a meta description

Format your HTML in a way that enables crawlers to better understand your app's content.

ADDITIONAL ITEMS TO MANUALLY CHECK (1) Show

Run these additional validators on your site to check additional SEO best practices.

PASSED AUDITS (6) Show

NOT APPLICABLE (3) Show

Captured at Nov 7, 2025, 11:05 PM GMT+5:30

Emulated Moto G Power with Lighthouse 12.8.2

Single page session

The application received the following scores:

Metric	Score	Analysis
Performance	**62 / 100**	**Needs Improvement.** The application is slow to load, even on a desktop.
Accessibility	**100 / 100**	**Excellent.** The app is fully accessible, with proper HTML structure and attributes.
Best Practices	**100 / 100**	**Excellent.** The app follows modern web development best practices.
SEO	**90 / 100**	**Good.** The app is well-structured for search engines but is missing a meta description.

4.2 Performance Analysis & Recommendations

While the application scores perfectly in Accessibility and Best Practices, the **Performance score of 62 is low**. This indicates a significant bottleneck in loading speed.

Key Performance Issues Identified (from Lighthouse Report):

- **Slow Load Time:** The report shows a **First Contentful Paint (FCP)** of **4.7 seconds** and a **Largest Contentful Paint (LCP)** of **5.6 seconds**. A good LCP is under 2.5 seconds.
- **Unused Code:** The report identified **88 KiB** of "unused JavaScript" and **41 KiB** of "un-minified JavaScript". This means the application is forcing the user to download code that is not immediately needed for the login page.
- **Inefficient Caching:** A "Use efficient cache lifetimes" warning was triggered, with **78 KiB** of resources that could be cached.

Actionable Recommendations:

1. **[SEO]:** Add a <meta name="description" content="..."> tag to the index.html file to fix the SEO score.
2. **[Performance]:** Implement **code-splitting** in the React app (using React.lazy() and Suspense) to only load the code for the Login page first, and then load other pages (like the Dashboard) on demand. This will directly address the "unused JavaScript" issue.
3. **[Performance]:** Configure browser caching policies for static assets in Vercel to fix the "efficient cache lifetimes" issue for repeat visitors.

5. Conclusion

The testing phase for MomentsHub v1.0 is complete.

- **Functionally,** the application is **stable and successful**. All core user stories (Login, Upload, Commenting, Tagging) passed E2E testing with Cypress.
- **Non-Functionally,** the application is **excellent in accessibility (100) and best practices (100)**. Its primary weakness is **Performance (62)**, which is a direct result of unoptimized, large JavaScript bundles.

The v1.0 branch is functionally complete. Future work in a v1.1 branch should focus on implementing the performance recommendations identified by Lighthouse to improve the application's load time.

6. Appendix

Appendix A: Functional Test Scripts (Cypress)

A.1 Test Script: login.cy.js:

```
// cypress/e2e/login.cy.js

describe('MomentsHub Login', () => {
  it('should allow a user to log in and see the dashboard', () => {
    // 1. Start the app (make sure it's running on port 5173)
    cy.visit('http://localhost:5173');

    // 2. Find the email input and type in it
    cy.get('input[type="email"]').type('test2@gmail.com');

    // 3. Find the password input and type in it
    cy.get('input[type="password"]').type('123456789');

    // 4. Find the submit button that contains "Sign In" and click it
    cy.get('button[type="submit"]').contains('Sign In').click();

    // 5. Assertion: Wait up to 10 seconds for the dashboard to load
    // We know it's the dashboard if we see "Today's Moments"
    cy.contains("Today's Moments", { timeout: 10000 });
  });
```

});

A.2 Test Script: upload.cy.js

```
// cypress/e2e/upload.cy.js

describe('Photo Upload Flow', () => {
  // Before each test, log the user in
  beforeEach(() => {
    cy.session('testUser', () => {
      cy.visit('http://localhost:5173');
      cy.get('input[type="email"]').type('test2@gmail.com');
      cy.get('input[type="password"]').type('123456789');
      cy.get('button[type="submit"]').contains('Sign In').click();
      // Wait for dashboard to ensure login is complete
      cy.contains("Today's Moments", { timeout: 10000 });
    });
  });

  it('should allow a user to upload a photo with a caption and tag', () => {
    // 1. Visit the dashboard
    cy.visit('http://localhost:5173');

    // 2. Click the '+' upload button
    cy.get('button[aria-label="Upload photo"]').click();

    // 3. Attach a test file
    // (This test will FAIL until you do Step 4 below)
    cy.get('input[type="file"]').selectFile('cypress/fixtures/test-image.png', { force: true });

    // 4. Type a caption
    const caption = `Cypress test photo @ ${new Date().toLocaleTimeString()}`;
    cy.get('textarea[placeholder="Add a caption to your moment..."]').type(caption);

    // 5. Add a tag
    cy.get('input[placeholder="Add a tag and press Enter"]').type('test-tag{enter}');

    // 6. Click the "Share Moment" button
```

```
    cy.get('button').contains('Share Moment').click();

    // 7. Assertion: Check that the new photo card appears in the feed
    cy.contains(caption);

    // 8. Assertion: Check that the tag is also visible
    cy.contains('#test-tag');
  });
});
```

A.3 Test Script: comment.cy.js

```
describe('Commenting Flow', () => {
  // Use the same login logic from our upload test
  beforeEach(() => {
    cy.session('testUser', () => {
      cy.visit('http://localhost:5173');
      cy.get('input[type="email"]').type('test2@gmail.com');
      cy.get('input[type="password"]').type('123456789');
      cy.get('button[type="submit"]').contains('Sign In').click();
      cy.contains("Today's Moments", { timeout: 10000 });
    });
  });

  it('should allow a user to add a comment to a photo', () => {
    // 1. Visit the dashboard
    cy.visit('http://localhost:5173');

    // 2. Find the first photo card and click it.
    // We use the class list from PhotoCard.jsx
    cy.get('.bg-white.dark\\:bg-gray-800.rounded-xl.shadow-md')
      .first()
      .click();
```

```
    // 3. Define a unique comment
    const commentText = `This is a test comment @ ${new Date().toLocaleTimeString()}`;

    // 4. Find the comment input, type in it.
    // We use :visible to get the one that isn't hidden (desktop vs. mobile)
    cy.get('input[placeholder="Add a comment..."]:visible').type(commentText);

    // 5. Find the submit button and click it
    cy.get('button[type="submit"]:visible').click();

    // 6. Assertion: The new comment text should now be visible on the page
    cy.contains(commentText);
  });
});
```